

Topic 8: Switching

CSE 4405: Data and Telecommunications

Md. Tanvir Hossain Saikat, Jr. Lecturer

July 17, 2026

Islamic University of Technology (IUT)

Where We Left Off — From a Shared Link to a Network of Links

Topic 7 recap

Multiple access answered: when *many stations share one medium*, who transmits?

ALOHA, CSMA/CD, CSMA/CA, polling, and channelization all **arbitrate one shared link**.

- One shared medium does not scale: stations may be **far apart**, and too many talkers destroy throughput (Pure ALOHA peaks near 18.4%).
- Yet a private wire per pair means $\binom{N}{2}$ links for N devices.

The big shift

One shared link is a **crowded room** where everyone shouts and collides.

Switching adds hallways and officers: a mesh of **switches** interconnects many links so any two devices talk *without* sharing one contended medium.

Topic 7 shared *one* link among many; Chapter 8 connects *many* links into a network.

Where Switching Sits in the Course

This chapter is the next rung on a ladder we have been climbing:

Lecture	Layer / role	Question it answers
Topic 6 — Multiplexing	Physical layer	How do several signals <i>share one link's capacity</i> ? (FDM/TDM)
Topic 7 — Multiple access	Data-link (lower)	<i>Who</i> may use one <i>shared</i> medium right now?
Topic 8 — Switching	Phys. / link / network	How do we <i>interconnect many links</i> so any source reaches any destination?

The thread continues

Circuit switching **reuses** the FDM and TDM channels of Topic 6 to divide each link, and contrasts with Topic 7's channelization (FDMA, TDMA, CDMA): instead of dividing *one* link among users, switching strings *many* links together. Routing and data-link control come next.

Learning Objectives

1. Explain what a switch is and how switched networks are classified
2. Describe circuit-switched networks: the three phases, efficiency, and delay
3. Explain datagram networks, routing tables, and on-demand resource use
4. Explain virtual-circuit networks: addressing, VCI, the three phases, and delay
5. Compare circuit, datagram, and virtual-circuit switching at the right protocol layer
6. Describe the structure of circuit switches (space-, time-, and combination switching)
7. Describe the structure of packet switches (ports, routing processor, switching fabric)

Source focus: Forouzan, *Data Communications and Networking*, Chapter 8 (Switching).

Chapter Roadmap

1. Circuit-switched networks

- Three phases, efficiency, and delay

2. Datagram networks

- Routing table, efficiency, and delay

3. Virtual-circuit networks

- Addressing, three phases, efficiency, and delay

4. Structure of a switch

- Structure of circuit switches and packet switches

Flow for this chapter: reserve a path first (circuit) → forward packets on demand (datagram) → blend both (virtual circuit) → look inside the switch hardware.

What Is Switching?

Switching (definition)

Switching is the technique of moving data from a source to a destination by **relaying it through intermediate nodes**, where each node directs incoming data toward the output that leads onward — a connection or forwarding decision made *on demand*.

The job of a node

Receive data on an **input** link, decide the correct **output** link, and forward it on — repeated at each node until the data reaches its destination.

Three ways to do it

This chapter studies three switching techniques — **circuit**, **packet** (data-gram), and **virtual-circuit** — that differ in *when* and *how* these connections are made.

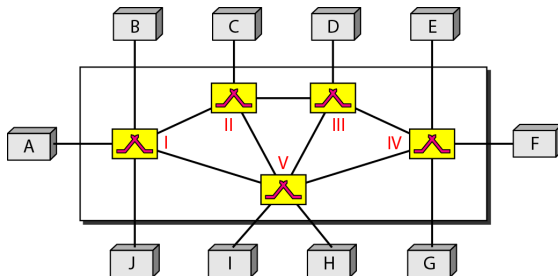
Switches and Switched Networks

Definition

Switches create temporary connections between the devices linked to them. A **switched network** is a series of interlinked **nodes** called **switches** (*general switching nodes, not Layer-2 LAN switches — see next slide*).

- End systems (A–J) connect through a mesh of switches (I–V).
- Each switch receives data on an input and forwards it toward the correct output.
- Only some pairs talk at once, so links are shared, not dedicated everywhere.

Figure: A switched network (Figure 8.1)



Terminology — “Switch” / “Node” Here vs. the Layer-2 LAN Switch

Watch out

In this chapter a **switch** (diagrams also call these interlinked **nodes**) means a *general switching node* in a usually **wide-area** network — *not* the Layer-2 Ethernet “switch” you plug computers into.

	“Switch” / “node” here	Layer-2 (LAN) switch
What it is	A generic switching node	A specific device
Concrete form	Telephone exchange, router , or ATM/Frame-Relay switch	Ethernet switch in a room/building
Forwards using	Reserved path, dest. address, or VCI	MAC address
Layer	Physical / network / data link	Data link
Scope	Wide-area network (WAN)	One local network (LAN)

So a *datagram* switch is really a **router**; the “nodes” (I–V) in Figure 8.1 are general switching nodes, not LAN switches.

Taxonomy of Switched Networks

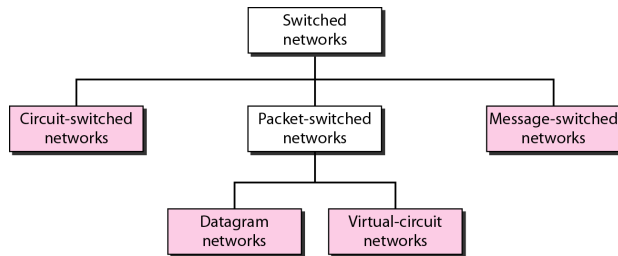
Three broad categories

Today's networks divide into **circuit-switched**, **packet-switched**, and **message-switched** networks.

Refinement

Packet switching has two subcategories: **datagram** networks and **virtual-circuit** networks. Message switching is mentioned only as historical context.

Figure: Taxonomy of switched networks (Figure 8.2)



8.1 Circuit-Switched Networks

Definition

A **circuit-switched network** consists of a set of switches connected by physical links. A connection between two stations is a **dedicated path** made of one or more links. However, each connection uses only one dedicated channel on each link.

- Each link is normally divided into n channels using **FDM** or **TDM**.
- Classic example: circuit-switched technology in the telephone network.

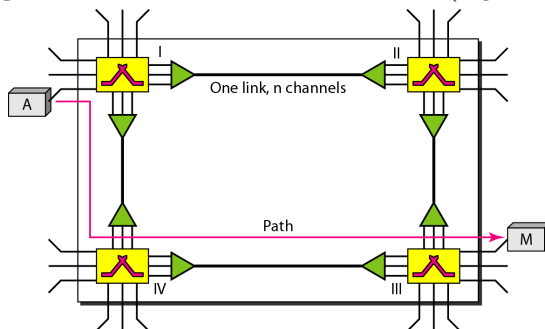
Topics in this section: three phases, efficiency, and delay.

A Trivial Circuit-Switched Network

- Four stations connect to four switches that define links between them.
- A path through the switches is reserved end to end before data flows.
- Each link carries the connection on one of its n channels.

When A wants to talk to M, a dedicated channel is reserved on every link along the path.

Figure: A trivial circuit-switched network (Figure 8.3)



Circuit Switching — The Three Phases

When end system A needs to communicate with system M, the connection passes through three phases:

- **Setup phase:** A requests a connection to M that must be accepted by all switches and by M itself. A channel is reserved on each link, and the combination of channels defines the dedicated path.
- **Data transfer phase:** once the dedicated path of connected channels is established, data transfer takes place.
- **Teardown phase:** after all data have been transferred, the circuits are torn down.

Note

In circuit switching, resources are reserved during the setup phase and remain dedicated for the entire duration of data transfer until the teardown phase.

The Three Phases — A Concrete Example

Scenario

Station **A** places a telephone call to station **M**. The path runs A—switch I—switch IV—M, and each link is divided into channels by **TDM** (numbered time slots).

1. **Setup.** A dials M. Switch I reserves a free slot (say **slot 4**) on link I→IV and forwards the request; switch IV reserves a free slot (say **slot 2**) on link IV→M. M accepts and an acknowledgment returns — now a dedicated channel exists on *every* link: A—(s4)—IV—(s2)—M. *(once, ~1–2 s)*
2. **Data transfer.** A and M talk. Voice flows continuously over the reserved 64 kbps channel both ways — **no routing decisions, no queueing**, constant minimal delay. The slots stay reserved even during silences.
3. **Teardown.** A hangs up. A teardown signal travels the path; switch I frees slot 4 and switch IV frees slot 2, returning them to the pool for other calls.

Resources are committed end-to-end *before* any data flows and released only *after* the call — the defining behaviour of circuit switching.

Circuit Switching — Efficiency and Delay

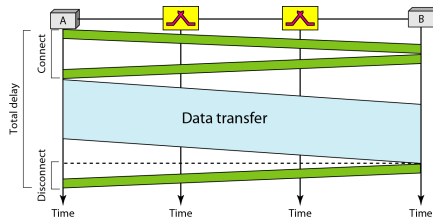
Efficiency

- Less efficient than the other two types: resources stay allocated for the *entire* connection.
- Those resources are unavailable to other connections meanwhile.

Delay

- Despite low efficiency, the delay is *minimal*.
- Data are not delayed at each switch; resources are pre-allocated for the whole connection.

Figure: Delay in a circuit-switched network (Figure 8.6)



Note

Switching at the **physical layer** in the traditional telephone network uses the circuit-switching approach.

Circuit Switching — Intuition

Analogy: a private highway lane

Setting up a circuit is like reserving a **private lane** from your driveway to your destination before you leave. Once reserved, you drive straight through — **no stopping** at any intersection, **no decisions** at each junction, guaranteed travel time.

The upside

Predictable, constant delay and bandwidth — ideal for a steady stream like a voice call.

The catch

The lane is **yours even while you idle**. If you stop for lunch, no one else may use it. Reserved-but-unused capacity is pure waste.

This “reserve the whole path” idea is exactly why circuit switching gives minimal delay but poor efficiency for bursty traffic.

Worked Example — Why Reservation Wastes Capacity

Setup

A telephone circuit reserves a dedicated 64 kbps channel for the *entire* call, yet each speaker is typically active only $\sim 40\%$ of the time (silence and listening fill the rest).

$$\begin{aligned}\text{Utilization} &= \frac{\text{capacity used}}{\text{capacity reserved}} \\ &= \frac{0.40 \times 64}{64} = 40\%\end{aligned}$$

- $\sim 60\%$ of the bandwidth is **paid for but idle**.
- It cannot be lent to another call during silences.
- For *bursty* data, utilization can fall to a few percent.

Takeaway

Reservation trades **efficiency** for **guarantees**; bursty traffic instead wants resources allocated *on demand* — the idea behind packet switching.

Worked Example — When Setup Dominates

The time to deliver a message over a circuit has three parts:

$$T_{\text{total}} = T_{\text{setup}} + \frac{L}{R} + T_{\text{teardown}}$$

L = message size, R = channel rate, plus a one-time setup and teardown

Large transfer (good fit)

$L = 50 \text{ MB}$, $R = 10 \text{ Mbps}$,
setup + teardown = 0.5 s.

$$T_{\text{total}} = 0.5 + \frac{50 \times 8}{10} \text{ s} = 0.5 + 40 = \mathbf{40.5 \text{ s.}}$$

Setup is $\sim 1\%$ — negligible.

Tiny message (bad fit)

$L = 1 \text{ kB}$, same R and setup.

$$T_{\text{total}} = 0.5 + \frac{8 \times 10^{-3}}{10} \text{ s} \approx 0.5 + 0.0008 \approx \mathbf{0.5 \text{ s.}}$$

$> 99\%$ of the time is **setup overhead!**

Reserving a whole path to send a few bytes is wasteful — this is precisely where datagram (packet) switching wins.

8.2 Datagram Networks

Idea

To send a message across a **packet-switched** network, it is divided into **packets** of fixed or variable size. The packet size is determined by the network and the governing protocol.

- Example: datagram networks in the Internet.

Topics in this section: routing table, efficiency, and delay.

Note

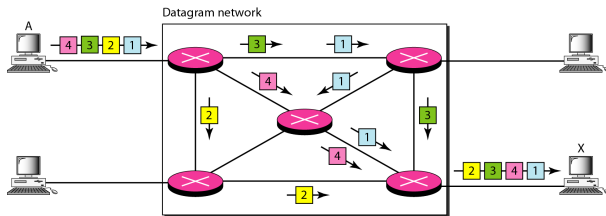
In a packet-switched network, there is **no resource reservation**; resources are allocated **on demand**.

Datagram Network

Independent packets

In a datagram network, each packet is treated **independently** of all others. Even if a packet is part of a multipacket transmission, the network treats it as though it existed alone.

Figure: A datagram network with four switches / routers (Figure 8.7)



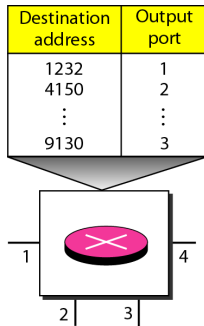
Routing Table

- A switch in a datagram network uses a **routing table** that is based on the **destination address**.
- The destination address in the packet header remains the *same* during the entire journey of the packet.

Efficiency

A datagram network is more efficient than a circuit-switched network: resources are allocated only when there are packets to be transferred.

Figure: Routing table in a datagram network (Figure 8.8)



Datagram Networks — Delay

- There may be *greater* delay in a datagram network than in a virtual-circuit network.
- Each packet may wait in a queue before being processed at a switch.

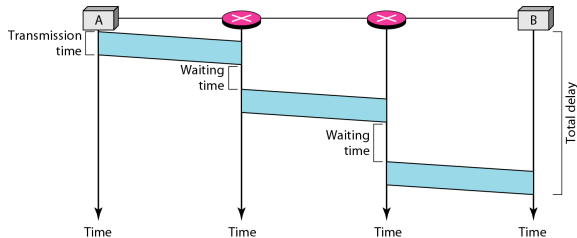
$$\text{Total delay} = 3T + 3\tau + w_1 + w_2$$

where T = transmission time, τ = propagation delay, w = waiting time

Note

Switching in the Internet is done using the **datagram** approach to packet switching at the **network layer**.

Figure: Delay in a datagram network (Figure 8.9)



Datagram Switching — Intuition

Analogy: independent postcards

Each packet is a self-addressed **postcard**. You drop several in the mailbox; the postal system looks at each one's *destination* and forwards it independently. No reservation, no promise about *which* route or *what order* they arrive in.

Consequences (the good)

No setup delay; a switch failure just reroutes later postcards; idle senders cost nothing — resources are used **on demand**.

Consequences (the cost)

Postcards may take **different paths**, suffer **variable queueing delay**, and arrive **out of order** — the receiver must re-assemble.

Worked Example — Tracing a Datagram by Destination

A packet for destination **1322** starts at switch 1. Each switch consults its own table, keyed only by the **destination address** (unchanged along the way), and emits the packet on the chosen output port.

Path found

S1 $\xrightarrow{\text{port 3}}$ S2 $\xrightarrow{\text{port 1}}$ S4 $\rightarrow$ host **1322**.

Nothing was reserved; the same destination just happens to map to a consistent next hop at each switch it visits.

Switch 1 table

Destination	Output port
1232	1
1322	3
1500	2

Switch 2 table

Destination	Output port
1320	2
1322	1
1900	4

Worked Example — Datagram Delay

For a packet crossing the 2-switch path of Figure 8.9 (3 transmissions, 3 propagations, queueing at each switch):

$$\text{Total delay} = 3T + 3\tau + w_1 + w_2$$

Plug in numbers

Transmission $T = 1$ ms, propagation $\tau = 2$ ms, queue waits $w_1 = 3$ ms, $w_2 = 1$ ms.

$$\begin{aligned} &= 3(1) + 3(2) + 3 + 1 \\ &= 3 + 6 + 4 = \mathbf{13\text{ ms}}. \end{aligned}$$

- The w_i terms depend on **current traffic** — they are not fixed.
- A later packet may see $w_1 = 8$, $w_2 = 0$ and a *different* total.
- Variable waits $\Rightarrow$ variable, possibly reordered, arrivals.

8.3 Virtual-Circuit Networks

Definition

A **virtual-circuit (VC) network** is a cross between a circuit-switched network and a datagram network. It has some characteristics of both.

Characteristics:

- As in a circuit-switched network, **there are setup and teardown phases** in addition to the data-transfer phase.
- **Resources can be allocated during setup (like circuit switching) or on demand (like a datagram network).**
- As in a datagram network, **data are packetized** and each packet **carries an address in its header** (*local*, not global, jurisdiction).
- As in a circuit-switched network, **all packets follow the same path** established during the connection.
- A VC network is normally implemented in the **data link layer**; circuit switching is in the physical layer and datagrams in the network layer.

Virtual Circuits — Intuition

Analogy: phone the route once, then mail postcards

First **call ahead** to fix a single route (like circuit switching's setup holding a private lane, you then send **labeled postcards** (like datagrams) along that agreed route. You get the *order* and *predictability* of a circuit, but not the *flexibility* of datagrams.

```
### First: establish the route
Before sending actual data, the sender and
network first establish one logical path
through the network. For example:
A → R1 → R3 → R5 → B
That path is chosen during a **setup phase**.
This is similar to circuit switching because
the route is fixed before communication begins.
But unlike a real circuit, the links are not
necessarily dedicated exclusively to this
connection.
```

```
### Then: send packets along that same route
Once the route is established, the message is
divided into packets.
Every packet follows the same path:
A → R1 → R3 → R5 → B
The packets do not independently choose a route
like ordinary datagrams do.
That is why delivery is usually **in order**.
```

Borrowed from circuits

Setup/teardown phases; one fixed path;
in-order delivery; optional resource reservation.

Borrowed from datagrams

Data is packetized; each frame carries a
small **label** (VCI), not a full global address.

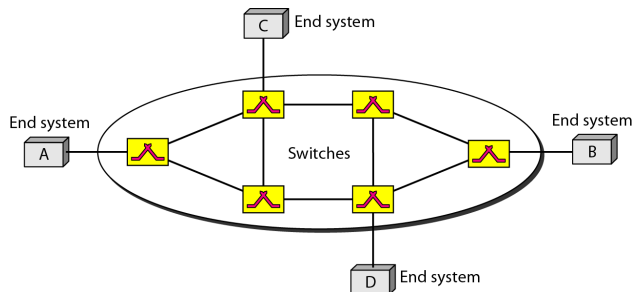
This is why a VC network is described as a *cross* between circuit switching and datagram switching.

Virtual-Circuit Network

- A VC network has switches that allow traffic from sources to destinations.
- A source or destination can be a computer, a packet switch, or another device that connects another network.

Switches in a WAN forward framed packets along a single pre-established virtual path.

Figure: Virtual-circuit network (Figure 8.10)



Addressing — Global Address and VCI

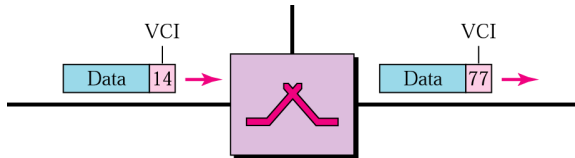
Global address

A **global address** is unique in the scope of the WAN or international network. In VC networks it is used only to *create* a virtual-circuit identifier.

Virtual Circuit Identifier (VCI)

A small number with only **switch scope**, used by a frame between two switches and actually used for **data transfer**. Each switch can use its own unique set of VCIs.

Figure: A VCI changes from link to link



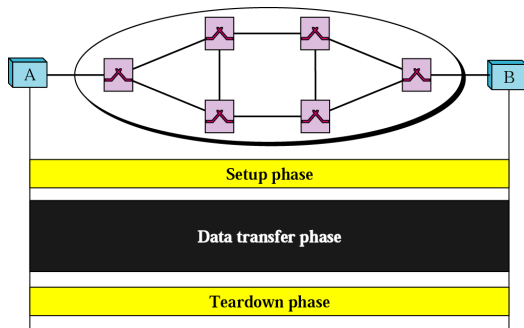
The same frame enters with VCI 14 and leaves with VCI 77
— the identifier is local to each link.

The Three Phases

To communicate, a source and destination go through three phases: **setup**, **data transfer**, and **teardown**.

- In the **setup** phase, the source and destination use their *global* addresses to help switches make table entries for the connection.
- In the **teardown** phase, the source and destination inform the switches to erase the corresponding entry.
- The **data** phase occurs between these two phases.

Figure: Setup, data transfer, and teardown phases

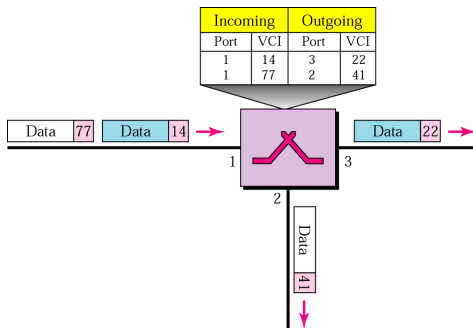


Data Transfer Phase

- To transfer a frame from source to destination, *all* switches need a table entry for this virtual circuit.
- Each switch maps an incoming (port, VCI) pair to an outgoing (port, VCI) pair.

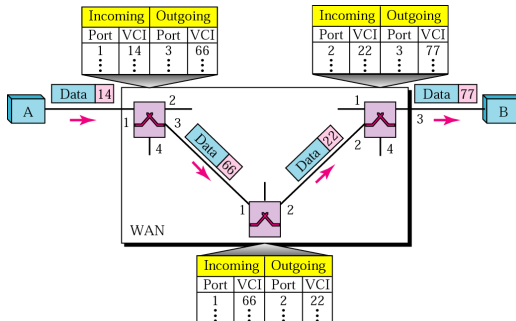
The VCI is swapped at every hop using the switch's table.

Figure: Switch table and frame relabeling



Data Transfer Phase (cont'd)

Figure: Source-to-destination data transfer across the WAN



Frame A enters with VCI 14; tables at each switch relabel it (14→66→22→77) so it follows one fixed path to B.

Worked Example — VCI Swapping Across Switches

A frame from A to B keeps *no* global address while travelling; each switch rewrites the small local **VCI** using its table entry (in port, in VCI) → (out port, out VCI).

Switch	In port	In VCI	Out port	Out VCI
S1	1	14	3	66
S2	2	66	3	22
S3	1	22	4	77

Label path: 14 → 66 → 22 → 77

Why relabel?

A VCI only needs to be unique on **one link between two switches**, so each switch may pick its own. Swapping keeps the labels small and lets switches assign VCIs independently.

The receiver sees VCI 77, which (together with the port) tells B the frame belongs to the A–B circuit, not some other source.

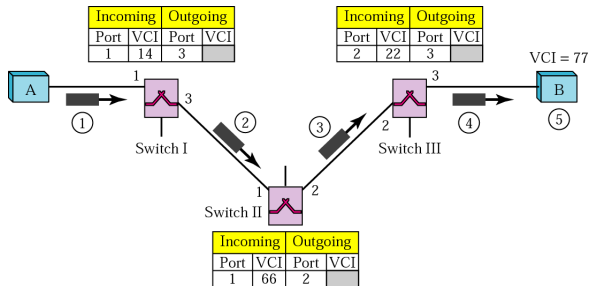
Setup Phase — Switched Virtual Circuit (SVC)

How a switch creates an entry

An **SVC** (temporary, per-session circuit) is created in two steps: a **setup request** ($A \rightarrow B$) and an **acknowledgment** ($B \rightarrow A$). The request carries the *global* addresses of A and B.

- Each switch picks the **outgoing port** (routing) and an unused **incoming VCI** for the arriving link (I: 14, II: 66, III: 22).
- Entry so far: in port, in VCI, out port; the **out VCI** stays empty — the next node has not chosen yet.

Figure: Setup request



VCI 77 lets the destination know the frames come from A and not other sources.

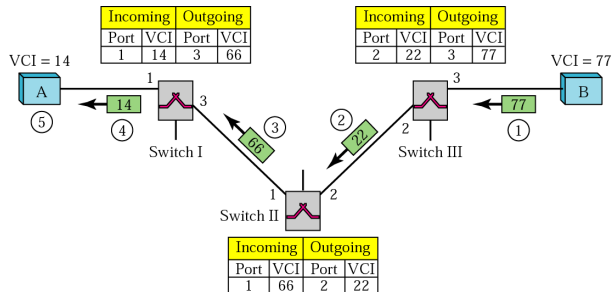
Setup Phase — SVC Acknowledgment

- B picks **77** as the label for frames delivered to it and sends it in the ack.
- Each ack hop carries the VCI that node picked during the request; the upstream switch copies it into its empty **out-VCI** column (III: 77, II: 22, I: 66).
- The final ack tells A to label its data frames with **14**.

The rule

On every link, the *receiver* picks the VCI (keeping its own incoming labels unique) and announces it *upstream* in the ack.

Figure: SVC setup acknowledgment



The ack travels back from B to A, completing the table entries along the chosen path.

Two Flavours of Virtual Circuit — PVC vs. SVC

Switched VC (SVC)

Set up **on demand** per session, then torn down — like **dialling a phone call** each time.

Permanent VC (PVC)

Pre-configured by the administrator, left *always up*; no per-session setup — like a **leased line**.

Aspect	SVC	PVC
Established by	Signalling, per call	Administrator, in advance
Setup/teardown delay	Yes, each session	None at use time
Best for	Occasional, dynamic pairs	Fixed, long-lived links

Frame Relay and ATM offered both: a branch office with constant traffic to HQ uses a **PVC**; ad-hoc connections use **SVCs**.

Teardown Phase and Efficiency

Teardown phase

- After sending all frames to B, source A sends a special frame called a **teardown request**.
- Destination B responds with a **teardown confirmation** frame.
- All switches **erase** the corresponding entry from their tables.

Efficiency (Note)

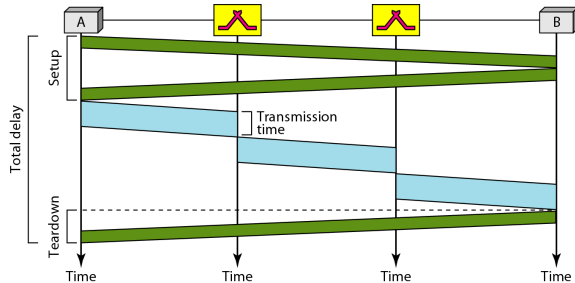
In virtual-circuit switching, all packets belonging to the same source and destination travel the **same path**; but packets may arrive with different delays if resource allocation is on demand.

Delay in Virtual-Circuit Networks

- There is a one-time delay for **setup** and a one-time delay for **teardown**.
- If resources are allocated during setup, there is *no* wait time for individual packets.

Total delay = $3T + 3\tau$ + setup delay + teardown delay

Figure: Delay in a virtual-circuit network



Note

Switching at the **data link layer** in a switched WAN is normally implemented using **virtual-circuit** techniques.

Worked Example — VC Delay vs. Datagram Delay

With resources reserved at setup, individual packets wait *nowhere*; the only extra costs are the one-time setup and teardown:

$$\text{Total delay} = 3T + 3\tau + T_{\text{setup}} + T_{\text{teardown}}$$

Same link, one packet

Using $T = 1$ ms, $\tau = 2$ ms as before, with $T_{\text{setup}} = 5$ ms, $T_{\text{teardown}} = 2$ ms:

$$= 3(1) + 3(2) + 5 + 2 = \mathbf{16\text{ ms}}$$
 (first packet).

A *later* packet on the same circuit: just $3T + 3\tau = \mathbf{9\text{ ms}}$ — **no** per-packet wait.

Compare

The datagram example paid $w_1 + w_2$ on *every* packet (variable, e.g. +4 ms). The VC pays setup *once*, then every packet is fast and **in order**.

Setup cost amortizes over many packets — great for long flows, wasteful for a single short one.

Virtual Circuits — The Whole Story on One Slide

Phase	Address used	What happens
1. Setup	Global addresses (used <i>only</i> here)	Request travels A→B choosing the path; each node picks an unused incoming VCI ; the ack travels back filling each outgoing VCI
2. Data transfer	VCI only (link-local labels)	Every frame is relabeled at every switch (14 → 66 → 22 → 77); all frames follow the same path, in order
3. Teardown	Global addresses	Source and destination tell the switches to erase the table entries

Recall hook

Phone the route once, then mail labeled postcards. Setup = the phone call (full names used); data = postcards carrying a short label that each post office swaps; teardown = cancelling the arrangement.

Circuit vs. Datagram vs. Virtual Circuit

Property	Circuit switched	Datagram	Virtual circuit
Path	Dedicated, reserved	Chosen per packet	Single, pre-established
Setup / teardown	Yes	No	Yes
Resource reservation	Whole connection	On demand	Setup or on demand
Address used to forward	Path set at setup	Destination address	Local VCI
Typical layer	Physical	Network	Data link

The virtual-circuit column borrows the phases of circuit switching and the packetization of datagrams.

Which Switching Should You Use?

If the workload is...	Choose	Because
Steady, real-time, constant rate (voice, leased video)	Circuit	Guaranteed bandwidth and minimal, constant delay outweigh low efficiency
Bursty, unpredictable, many destinations (web, email, Internet)	Datagram	On-demand resources adapt to bursts; survives link/switch failures
Long connection-oriented flow needing order/QoS (switched WANs: Frame Relay, ATM; MPLS today)	Virtual circuit	One-time setup buys ordered delivery with small labels (low jitter if resources are reserved)

Rule of thumb

Reserve a **circuit** for heavy, steady use; send **datagrams** for bursty, flexible traffic; build a **virtual circuit** for a circuit's order and predictability with a packet network's efficiency.

8.4 Structure of a Switch

Scope

Switches are used in both circuit-switched and packet-switched networks. This section discusses the **internal structure** of the switches used in each type of network.

Topics in this section:

- Structure of circuit switches
- Structure of packet switches

From Technique to Hardware — What Must Each Box Do?

Sections 8.1–8.3 chose *how* to move data. Now we open the box and ask what the switch itself must be built to do:

	Circuit switch (8.1)	Packet switch (8.2, 8.3)
Job	Hold a continuous path for the whole call	Make a fresh decision for every packet
Needs	Physically connect input i to output j and <i>keep</i> it	Queues, an address lookup, and a fast internal mover
Structure	Space division (crossbar, multistage) or time division (TSI)	Ports + routing processor + switching fabric

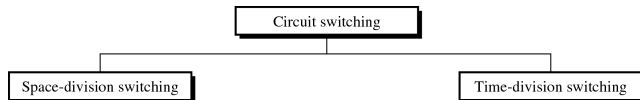
Both halves share one worry: connecting N inputs to N outputs cheaply. Watch the crossbar idea appear twice — and get replaced by a multistage design twice.

Structure of Circuit Switches

A circuit switch can use either of two technologies:

- **Space-division switching:** paths are separated from each other *spatially*.
- **Time-division switching:** paths are separated in *time* using TDM.

Figure: Two technologies for circuit switching



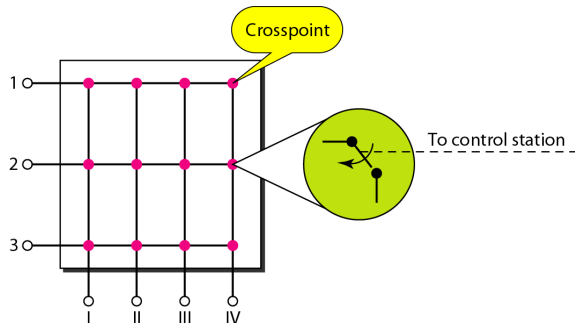
Space-Division Switching — Crossbar Switch

Crossbar switch

Connects n inputs to m outputs in a **grid**, using an electronic microswitch (transistor) at each **crosspoint**.

- Paths through the circuit are separated spatially.
- Any input can reach any free output by closing one crosspoint.

Figure: Crossbar switch with three inputs and four outputs (Figure 8.17)



Limitation of the Crossbar Switch

Huge size

Connecting n inputs to m outputs requires $n \times m$ crosspoints. For example, 1000 inputs and 1000 outputs need 1,000,000 crosspoints.

Inefficient

Fewer than 25% of the crosspoints are in use at any given time; the rest sit idle.

Why it matters

The cost and waste of a single huge crossbar motivate the **multistage** switch, which reuses smaller crossbars.

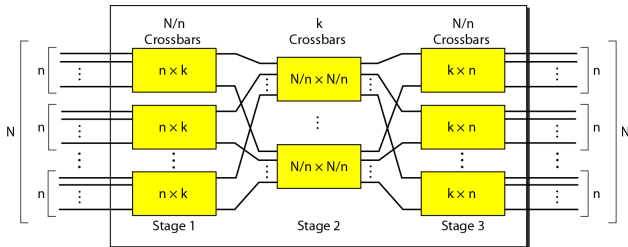
Multistage Switch

Multistage switch

Combines crossbar switches in several (normally **three**) stages to solve the size limitation of a single crossbar switch.

Inputs are grouped into small crossbars, linked through a middle stage, then regrouped — far fewer crosspoints.

Figure: Multistage switch (Figure 8.18)



Worked Example — Crosspoints Saved by Going Multistage

Split N inputs into groups of n : that gives N/n first-stage crossbars, k middle crossbars, and N/n third-stage crossbars. For a **nonblocking** design we need $k = 2n - 1$.

$$\text{Crosspoints} = \underbrace{2kN}_{\text{stages 1 \& 3}} + \underbrace{k \left(\frac{N}{n} \right)^2}_{\text{stage 2}}, \quad \text{with } k = 2n - 1$$

$N = 200$, **optimal** $n = 10 \Rightarrow k = 19$

Stages 1 & 3: $2(19)(200) = 7600$

Stage 2: $19 \times (200/10)^2 = 19 \times 400 = 7600$

Total = 15,200 crosspoints

Single crossbar: $200 \times 200 = \mathbf{40,000}$.

- $\sim 62\%$ fewer crosspoints — and the gap **widens** as N grows.
- Recall a single 1000×1000 crossbar needs 10^6 crosspoints; multistage slashes this.
- Cost: a multistage switch can *block* unless $k \geq 2n - 1$.

Why $k \geq 2n - 1$ Means No Blocking

Focus on **one** first-stage crossbar (mine) and **one** third-stage crossbar (my target). Each middle switch is a possible “hub” between them — when could *all* hubs be unusable?

Worst-case count

- The other $n - 1$ inputs of *my* group are busy, each holding a **different** middle switch.
 - The other $n - 1$ outputs of my *target* group are busy via $n - 1$ **further** middle switches.
- $\Rightarrow$ at most $2(n - 1) = 2n - 2$ hubs can be blocked.

One more hub wins

Provide $k = (2n - 2) + 1 = 2n - 1$ middle switches and *some* hub is always free — the switch can **never** block, however unlucky the traffic. Fewer hubs are cheaper but risk blocking; $2n - 1$ is the guaranteed-safe minimum (the *Clos* condition).

This is why the worked example used $k = 19$ for $n = 10$: $2(10) - 1 = 19$.

Time-Division Switching — TSI

Time-division switch

Uses **time-division multiplexing (TDM)** inside a switch. The most popular technology is the **time-slot interchange (TSI)**, which changes the ordering of the slots based on the desired connections.

Figure: TDM without a TSI

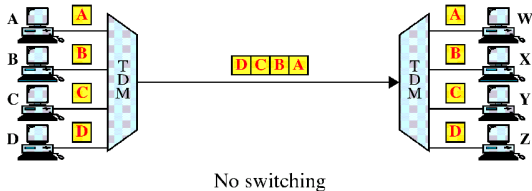
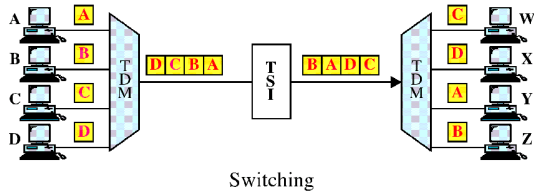
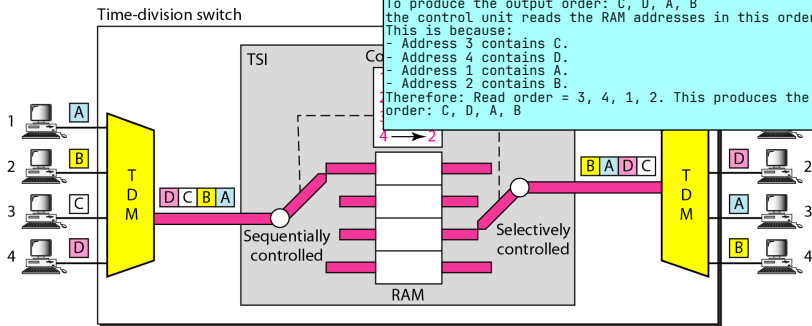


Figure: TDM with a TSI



Time-Slot Interchange (TSI)

Figure: Time-slot interchange



Step 1: Write Incoming Data Sequentially. The arriving data samples are stored in RAM in their original order:

RAM Address	Stored Data
1	A
2	B
3	C
4	D

This is called ****sequential writing**** because the RAM addresses are used in order: 1, 2, 3, 4

Step 2: Read the Data Selectively

The control unit determines the order in which the RAM addresses are read.

To produce the output order: C, D, A, B

the control unit reads the RAM addresses in this order: 3, 4, 1, 2

This is because:

- Address 3 contains C.

- Address 4 contains D.

- Address 1 contains A.

- Address 2 contains B.

Therefore: Read order = 3, 4, 1, 2. This produces the desired output-slot order: C, D, A, B

RAM stores incoming slots in order; the control unit reads them out in a new order (1→3, 2→4, 3→1, 4→2) to realize the desired connections.

Space vs. Time Division — The Trade-off

	Space division (crossbar)	Time division (TSI)
How paths differ	Different wires	Different time slots
Speed	Instantaneous — close a crosspoint and the path exists	Adds delay : a slot may wait in RAM up to one full frame
Cost	Expensive : crosspoints grow as N^2	Cheap : just memory and a control table

Each one's weakness is the other's strength

So real telephone switches **combine** them: cheap TSIs shuffle slots at the edges, and a small, fast space stage jumps between lines — the **TST** design on the next slide.

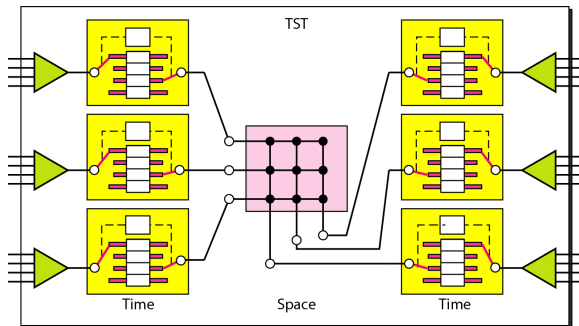
Space- and Time-Division Combinations

Combine the best of both

Combine space-division and time-division technology to take advantage of the strengths of each. Common arrangements:

- **TST** (time-space-time)
- **TSST** (time-space-space-time)
- **STTS** (space-time-time-space)

Figure: Time-space-time (TST) switch (Figure 8.20)

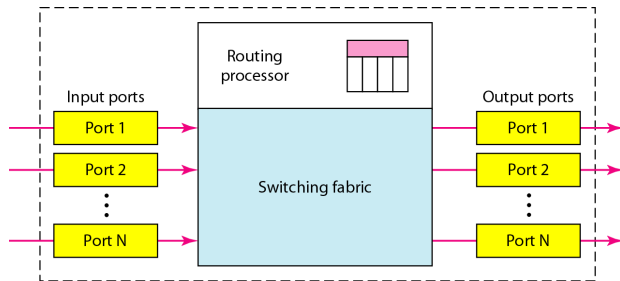


Structure of Packet Switches

Four components

A switch used in a packet-switched network has a different structure from a circuit switch. A packet switch has four components: **input ports**, **output ports**, the **routing processor**, and the **switching fabric**.

Figure: Packet switch components (Figure 8.21)



Input and output ports contain queues where packets wait when forwarding lookup, the switching fabric, or the outgoing link cannot serve them immediately.

The switching fabric may use a binary multistage structure in which each switching node forwards a packet through one of two outgoing paths. The destination-port bits guide the packet through successive stages until it reaches the required output port. Because only a small binary decision is made at each stage, packet transfer through the fabric is very fast.

Intuition — A Packet Switch Is a Post Office

Component	Post office	What it really does
Input port	Receiving dock	Physical + data-link work: signal → bits → frame, error check, then queue
Routing processor	Sorting supervisor	Network-layer work: look up the destination, pick the output port
Switching fabric	Conveyor belts	Move the packet from input queue to output queue — the <i>hard, fast</i> part
Output port	Shipping dock	Queue, re-frame (data link), bits → signal

Only the routing processor is “smart”; the rest is about moving packets fast without losing them. Queues appear at both ends because arrivals are bursty — a packet-world problem a circuit switch never has.

Input and Output Ports

Figure: Input port (Figure 8.22)

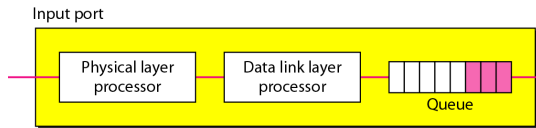
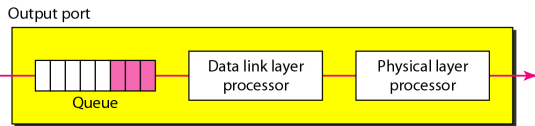


Figure: Output port (Figure 8.23)



An input port performs the physical and data link functions of the packet switch.

An output port performs the same functions as the input port, but in the reverse order.

Routing Processor and Switching Fabric

Routing processor

- Performs the functions of the **network layer** (looks up the destination and decides the output port).

Switching fabric

- The most difficult task in a packet switch is to move a packet from the *input* queue to the *output* queue.
- Modern packet switches use specialized mechanisms with a variety of switching fabrics.
- The **crossbar switch** is the simplest type of switching fabric.

Fabric Choice — Why (Not) Just Use a Crossbar?

Why the crossbar is the first choice

- **Nonblocking**: packets bound for *different* outputs cross simultaneously without touching.
- **Fast and direct**: close one crosspoint and the packet is across — no multi-hop journey inside the switch.
- **Conceptually trivial**: the simplest switching fabric there is.

Why it does not scale to big routers

- $N \times N$ ports need N^2 crosspoints ($8 \times 8 = 64$; $1000 \times 1000 = 10^6$) — the same wall as in circuit switching.
- A **central scheduler** must pick a conflict-free set of crosspoints *every packet cycle*, millions of times per second — the scheduler becomes the bottleneck.
- Two packets to the *same* output still contend, so queues remain anyway.

Wanted: a fabric with fewer elements and *no central brain*. The self-routing **banyan** (next) needs only $\frac{N}{2} \log_2 N = 12$ microswitches for 8×8 , versus 64 crosspoints.

Banyan Switch

Banyan switch

A multistage switch with microswitches at each stage that route packets based on the **output port** represented as a **binary string**.

Figure: A banyan switch (Figure 8.24)

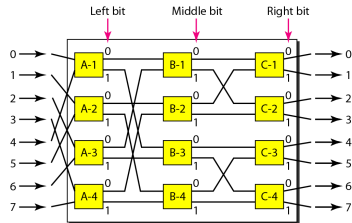
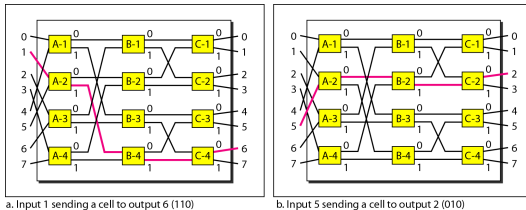


Figure: Examples of routing in a banyan switch (Figure 8.25)



Drawing Any Banyan — Step 1: The Skeleton Is Forced by Arithmetic

Goal: a recipe that draws a correct $N \times N$ banyan for *any* $N = 2^n$ — no memorising of Figure 8.24.

Three numbers fix the whole skeleton

- **Columns:** $n = \log_2 N$ stages — *one per destination bit*.
- **Rows:** $N/2$ microswitches per column — each 2×2 box terminates two of the N lines.
- **Labels:** outputs $0 \dots N-1$ top-to-bottom as n -bit strings $b_1 \dots b_n$; at every switch, bit **0** $\Rightarrow$ **upper** output, bit **1** $\Rightarrow$ **lower**.

N	stages	sw. per stage	total
8	3	4	12
16	4	8	32
1024	10	512	5120

Why $\log_2 N$ columns is not a design choice

A 2×2 microswitch answers exactly **one yes/no question**: “top or bottom half of the outputs I can still reach?” Finding 1 output among N takes $\log_2 N$ halvings — a banyan is **binary search built out of hardware**, so the column count is forced, not chosen.

Skeleton done: $\log_2 N$ columns $\times$ $N/2$ rows of empty boxes. All the cleverness is in the *wiring* — one recursive rule, next slide.

Drawing Any Banyan — Step 2: One Wiring Rule, Applied Recursively

The rule (start with the whole switch, $M = N$)

1. **Shuffle in:** **top-half** input $k \rightarrow$ **upper** port of switch k ; **bottom-half** input $k \rightarrow$ **lower** port — the halves *interleave*, like a card riffle.
2. **Route one bit:** the column reads the next destination bit: $0 \Rightarrow$ upper output, $1 \Rightarrow$ lower.
3. **Split and recurse:** the **upper** outputs (top-to-bottom) feed a **top** $\frac{M}{2}$ -block, the **lower** outputs a **bottom** one; recurse until $M = 2$ — the last column.

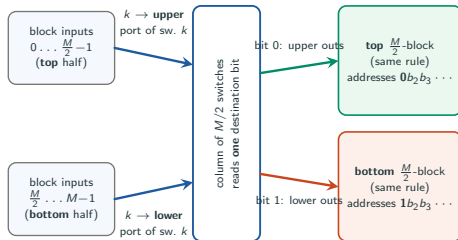


Figure 8.24 is this rule applied three times — next slide.

The logic behind each step

Step 3 is binary search made physical: addresses starting with 0 are exactly the top half of outputs, so after b_1 the same problem remains, one bit fewer. **Step 1's shuffle** is not needed for self-routing — but it deals neighbours onto *different* switches, which the Batcher-banyan exploits shortly.

Drawing Any Banyan — Step 3: The Recipe Rebuilds Figure 8.24 Exactly

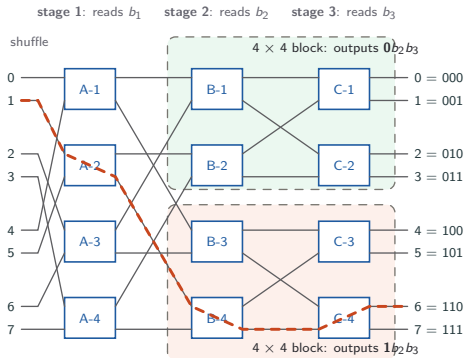
Three applications of the rule ($N = 8$)

$M = 8$: shuffle the 8 inputs into column A; upper outputs $\rightarrow$ green block, lower $\rightarrow$ coral.

$M = 4$ (twice): shuffle each block's 4 lines into its B-column; split again.

$M = 2$: the C-switches are the base case — outputs straight, stop.

This *is* Figure 8.24, wire for wire: the book's crossed input lines are the $M=8$ shuffle, and each tinted half is a visibly smaller banyan — the recursion on display.



Check your drawing

Count: $\frac{N}{2} \log_2 N = 12$ switches, 8 wires per gap. **Self-route:** follow 110 from input 1 (dashed): lower, lower, upper $\Rightarrow$ output 6 — Figure 8.25a reproduced. **Uniqueness:** any (input, output) pair has exactly *one* path.

Worked Example — Self-Routing in a Banyan Switch

In an 8×8 banyan there are 3 stages, and the output port is a 3-bit number. Each packet carries the destination in binary; **stage i reads bit i** (most-significant first): bit 0 sends the packet to the microswitch's **upper** output, bit 1 to the **lower**.

Route a packet to output $6 = (110)_2$

Stage 1 bit = **1** $\rightarrow$ lower

Stage 2 bit = **1** $\rightarrow$ lower

Stage 3 bit = **0** $\rightarrow$ upper

Packet arrives at output $(110)_2 = \mathbf{6}$, regardless of which input it started from.

Why “self-routing”

No central controller and no table: the **destination bits themselves** steer the packet stage by stage. Tiny microswitches just read one bit each.

Weakness: two packets can collide on a shared internal link — worked out on the next slide.

Worked Example — The Banyan's Weakness (Internal Collision)

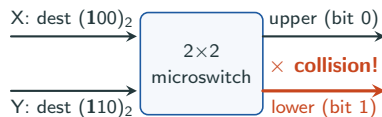
Same cycle, same first-stage microswitch: X wants output 4 = $(100)_2$, Y wants output 6 = $(110)_2$.

Stage 1 reads the first bit of each

X: dest 100 bit = 1 → lower

Y: dest 110 bit = 1 → lower

Both need the **same wire**, which carries one packet per cycle: one passes, the other is **dropped or buffered**.



The subtle point

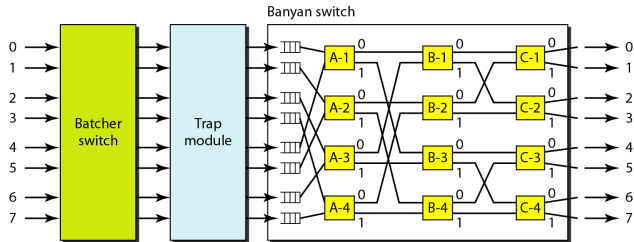
Outputs 4 and 6 are *different and both idle* — the fight is over a **shared internal link**. A banyan has exactly one path per (input, output) pair, so packets cannot detour. Fix: **sort** each batch by destination first — the **Batcher** stage, next slide.

Batcher-Banyan Switch

Batcher-banyan switch

Designed to solve the problem of the banyan switch. A **Batcher** stage comes before the banyan switch and **sorts** the incoming packets according to their final destination.

Figure: Batcher-banyan switch (Figure 8.26)



Inside the Batcher-Banyan — Sort, Trap, then Route

The Batcher sorter is a sorting network

Layers of two-input **comparators**: each compares the **destination numbers** of its two packets and sends the *smaller* to its upper wire, the *larger* to its lower wire. After the last layer the batch leaves **sorted by destination** — all in parallel hardware.

The trap module

Sorting has a bonus: packets with the *same* destination come out **side by side**. The **trap** module between sorter and banyan therefore only compares neighbours: **one** packet per destination proceeds; duplicates are **held for the next cycle**.

Why sorted input tames the banyan

A sorted batch enters **gap-free at the top**; the banyan's **input shuffle** (step 1 of the drawing rule) deals these neighbours onto *different* first-stage switches. Packets that still meet at a switch head for *different halves* — one takes the upper wire, one the lower. Internal collision is engineered away *before* entry.

Worked Example — One Cycle Through a Batch-Banyan

Same troublemakers as the collision slide — **X→4 (100)** and **Y→6 (110)** — plus **W→1 (001)** and **Z→6 (110)**, all arriving in one cycle of an 8×8 switch.

Stage		What happens
1.	Batcher sorts	Batch leaves sorted and gap-free on the top lines: W(001), X(100), Y(110), Z(110)
2.	Trap checks	Neighbours Y and Z both carry 110 → Z is held : an <i>output</i> conflict, caught <i>before</i> entry
3.	Banyan routes	The shuffle deals W, X, Y onto A-1, A-2, A-3 — X and Y no longer share a switch . X: A-2 → B-4 → C-3 → 4 ; Y: A-3 → B-3 → C-4 → 6 — disjoint wires at every stage
4.	Next cycle	Z re-enters at the front of the next batch

Why the old crash cannot recur

On the collision slide, X (100) and Y (110) met at *one* microswitch, both demanding its **lower** wire. The Batcher stage delivers a **sorted, compact** batch; the banyan's **input shuffle** spreads consecutive packets across *different* switches, so packets sharing a switch head for *different halves*. Internal collisions vanish *by construction*; the one conflict left — same *output* — was already removed by the trap.

Cost of the guarantee: extra sorting stages, and (rarely) a one-cycle wait for duplicate-destination packets.

8.4 in One Picture — Recall Map

	Circuit switch	Packet switch fabric
Simplest idea	Crossbar: N^2 crosspoints	Crossbar: same N^2 problem
Too big? Go multistage	Three stages of small crossbars	Banyan: self-routing by destination bits
New problem created	Blocking (if k too small)	Internal collisions
The fix	$k \geq 2n - 1$ middle switches	Batcher sorter (+ trap) before the banyan
Or switch in <i>time</i>	TSI reorders TDM slots in RAM; combine as TST	—

The chapter's rhyme

The **obvious** design (one big crossbar) is perfect but unaffordable; **multistage** is affordable but creates a new problem (blocking / collisions); a clever addition ($k = 2n - 1$ / Batcher sorting) buys the guarantee back.

Summary (1)

- A switched network consists of a series of interlinked nodes, called **switches** (general switching nodes — exchanges, routers, or VC switches, *not* Layer-2 LAN switches): circuit switching, packet switching, and message switching.
- Networks divide into three broad categories: **circuit-switched**, **packet-switched**, and **message-switched**. Packet-switched networks split further into **virtual-circuit** and **datagram** networks.
- A circuit-switched network is a set of switches connected by physical links. Resources are reserved during the **setup** phase and stay dedicated through data transfer until the **teardown** phase.
- In packet switching there is **no** resource allocation for a packet — no reserved bandwidth and no scheduled processing time. Resources are allocated **on demand**.

Summary (2)

- In a **datagram** network, each packet is treated independently. Packets are referred to as datagrams; there are *no* setup or teardown phases.
- A **virtual-circuit** network is a cross between a circuit-switched network and a datagram network, with characteristics of both.
- Circuit switching uses either the **space-division** switch or the **time-division** switch.
- A packet switch has a different structure from a circuit switch: four components — **input ports**, **output ports**, a **routing processor**, and the **switching fabric**.

Key Terminology (Forouzan Ch. 8)

Term	Lecture use in this chapter
Dedicated channel	One reserved channel per link for a circuit-switched connection
FDM / TDM channels	How a link is divided into n channels for circuit switching
Datagram	A packet treated independently, forwarded by destination address
Routing table	Destination-based forwarding table in a datagram switch
VCI	Local virtual-circuit identifier, swapped at each switch
SVC	Switched virtual circuit set up on demand for one session
Crossbar / crosspoint	Grid switch with a microswitch at each input-output intersection
TSI	Time-slot interchange that reorders TDM slots via RAM
Switching fabric	Mechanism that moves packets from input to output queues
Banyan / Batcher-banyan	Multistage packet fabrics that route (and pre-sort) by binary destination

Questions?

Discussion: when to reserve a circuit, when to send datagrams, and when a virtual circuit is the right compromise.